

Tutorial: Eine eigene Datenquelle anbinden

Für Entwickler:innen ohne Vorerfahrung · ~30 Minuten · läuft komplett gegen das Demo-Szenario · jede Etappe mit Beispiel

1 Das Modell in drei Begriffen

Alles im sources-Framework dreht sich um drei Dinge. **Record**: das normierte Ergebnis — der Report sieht immer nur Records, nie dein System. Für ein SLO:

```
{"service": "Order API", "slo": "availability",
 "target_pct": 99.9, "sli_pct": 99.95,
 "window": "30d", "source": "csv:slos.csv"}
```

Provider: ein Übersetzer — er liest dein System (Datei, REST-API, was auch immer) und gibt Records zurück; eine Python-Datei, eine Klasse. **Register**: die JSON-Datei, die ein Abruf schreibt; die Solution-Config referenziert sie ("slo": "slo.json"), der Report rendert sie. Eine Regel hält alles vergleichbar: **Provider urteilen nie** — ob ein SLO breached ist, entscheidet eine zentrale Regel, dieselbe für jede Quelle.

2 Erst benutzen, dann bauen (5 Minuten)

Erzeuge das Demo-Portfolio, sieh dir die Provider-Liste an und mache deinen ersten Abruf mit dem file-Provider — das Ergebnis (my_slo.json) ist das Zielformat jeder Quelle:

```
python -m testdata_generator --scenario portfolio --output demo --seed 42
python -m sources providers

echo {"provider": "file", "path": "demo/slo_alpha.json"} > my_sources.json
python -m sources fetch --kind slo --config my_sources.json --output my_slo.json
```

3 Deinen eigenen Provider bauen, Schritt für Schritt

Wir binden eine CSV-Datei an (Teams pflegen SLO-Werte oft in einer Tabelle) — kein Server nötig, das Muster ist für jedes System identisch. Die fertige Referenz liegt in sources/providers/csv_slo.py. **Schritt 1**: Lege EINE Datei an, sources/providers/my_csv.py:

```
from __future__ import annotations

import csv
from datetime import datetime
from pathlib import Path

from sources.base import KIND_SLO, SloRecord

class MyCsvSource:
    # (1) Name in Configs und in der providers-Liste / name in configs
    provider_id = "my_csv"
    # (2) Was diese Quelle liefern kann / what it can deliver
    kinds = (KIND_SLO,)

    # (3) Der Uebersetzer / the translator
    def fetch(self, kind, config, log):
        path = Path(config["path"])
        if not path.is_file():
            raise RuntimeError(f"my_csv: '{path}' missing.")
        fetched_at = datetime.now().isoformat(timespec="seconds")
        records = []
        with path.open(encoding="utf-8-sig", newline="") as f:
            for row in csv.DictReader(f):
                records.append(SloRecord(
                    service=row["service"],
                    slo=row["slo"],
                    target_pct=float(row["target_pct"]),
                    sli_pct=float(row["sli_pct"]) if row.get("sli_pct") else None,
                    source=f"my_csv:{path.name}",
```

```

        fetched_at=fetched_at,
    ))
    log(f"  my_csv: {len(records)} records")
    return records

```

(4) Genau dieses Objekt sucht die Auto-Discovery / discovery hook
 PROVIDER = MyCsvSource()

Vier nummerierte Ideen sind der ganze Contract: Id, Arten, ein fetch(), das übersetzt, und ein PROVIDER-Objekt. Nirgends ist ein Register zu pflegen — die Existenz der Datei ist die Registrierung. **Schritt 2:** python -m sources providers → „my_csv: slo“ erscheint. **Schritt 3:** Daten geben (team_slos.csv):

```

service,slo,target_pct,sli_pct
Order API,availability,99.9,99.95
Checkout,availability,99.5,99.55

```

Schritt 4 und 5: Abruf-Config anlegen, fetchen, das Register in der Solution-Config eintragen ("slo": "my_slo.json") und den Report rendern — die Sektion „Service Levels & Error Budgets“ zeigt deine CSV-Zeilen, die Spalte Data source sagt my_csv:team_slos.csv:

```

{"provider": "my_csv", "path": "team_slos.csv"}

python -m sources fetch --kind slo --config csv_source.json --output my_slo.json
python -m portfolio demo/solution_alpha.json --output report.html --browser

```

4 Ein REST-System statt einer Datei?

Gleiches Muster, nur fetch() ändert sich — der gemeinsame Helfer übernimmt HTTP, Auth und Fehler-Mapping:

```

from sources.http import bearer, get_json, token_from_env

def fetch(self, kind, config, log):
    headers = bearer(token_from_env(config, "MY_SYSTEM_TOKEN"))
    data = get_json(config["base_url"] + "/api/slos", headers, "MySystem")
    return [SloRecord(service=e["name"], slo=e["goal"],
                      target_pct=e["target"], sli_pct=e["current"],
                      source="my_system") for e in data]

```

- Tokens nur aus Umgebungsvariablen (token_from_env) — nie in Configs, nie in Logs.
- get_json mappt Fehler: 401/403 verweist auf die womöglich fehlende API-Freigabe UND den Datei-Weg als Ausweichroute.
- Mit Mocks testen, nicht gegen das Live-System — Request-Recorder-Muster aus tests/sources/unit/test_rest_providers.py kopieren.

5 Eine Quelle austauschen

Austauschen heißt: die Abruf-Config ändern, sonst nichts. Register-Name, Solution-Config und Report bleiben unverändert — nur die Data-source-Spalte zeigt die neue Herkunft:

```

# vorher / before:
{"provider": "my_csv", "path": "team_slos.csv"}

# nachher / after (gleicher fetch-Befehl, gleiches Register):
{"provider": "prometheus", "base_url": "https://prom.intern",
 "services": [{"service": "Order API", "slo": "availability",
                  "target_pct": 99.9,
                  "sli_query": "avg_over_time(up[30d])", "scale": 100}]}

```

6 Zwei Quellen kombinieren

Eine Config darf eine sources-Liste tragen; die Records fließen in EIN Register, jede Zeile behält ihre Herkunft. Typisch: das meiste im Monitoring, zwei Altsysteme in einer Tabelle:

```

{"sources": [
  {"provider": "prometheus", "base_url": "https://prom.intern",
   "services": [ ... ]},
  {"provider": "my_csv", "path": "team_slos.csv"}
]

```

}}

7 Quellen im Alltag verwalten

- Inventar: `python -m sources providers` ist die maßgebliche Liste — neue Datei erscheint automatisch.
- Tokens: eine Umgebungsvariable je System (`PROMETHEUS_TOKEN`, `GITHUB_TOKEN`, `GITLAB_TOKEN`, `SONAR_TOKEN`; eigene via `token_env`).
- Konventionen: eine Datei je Quelle; Provider übersetzen, urteilen nie; nicht Gemessenes ist `None`, keine Schätzung.
- Fehler: 401/403 nennt den Freigabe-Hinweis — bis dahin dieselben Daten über `file` oder `csv` liefern.
- Tests: kleine Testdatei je Provider (Happy Path, eine kaputte Eingabe, lesbare Fehlermeldung).

8 Checkliste für eine neue Quelle

- Eine Datei in `sources/providers/` mit `provider_id`, `kinds`, `fetch()`, `PROVIDER` — `providers` listet sie.
- `fetch()` liefert normierte Records; jeder Record setzt `source`; kein Urteil im Provider.
- Tokens via `token_from_env`; Mock-Tests grün; Rücktausch auf `file/csv` funktioniert (die Ausweichroute bei ausstehender Freigabe).